Griffin: Customizable Data and Query Generation for Filtered ANNS Benchmarking

Zirui Hu Hailin He Pinshan Cao Kai Huang Rong Zhang Peng Cai Chengcheng Yang Xuan Zhou
School of Data Science and Engineering, East China Normal University
{zrhu, hlhe, caopinshan}@stu.ecnu.edu.cn, kylehuangk@gmail.com, {rzhang, pcai, ccyang, xzhou}@dase.ecnu.edu.cn

***Abstract*—Filtered approximate nearest neighbor search (FANNS) is widely used in modern retrieval systems. Comprehensive evaluation of FANNS requires customizable control over attribute–vector dependencies and query hardness, characterized by *selectivity* and *query–filter correlation* (i.e., how filtered vectors are distributed around the specified query vector). However, achieving such control faces two challenges: 1) traditional tuple-level alignment to achieve attribute–vector dependencies yields a search space of $N!$ assignments for N tuples; 2) instantiating queries to meet a target hardness suffers from a combinatorial search space and costly validation. To address this, we propose GRIFFIN, a customizable framework that jointly generates attribute–vector datasets and FANNS workloads satisfying user-specified data dependencies and query hardness targets. First, we reformulate attribute–vector dependency control from tuple-level alignment into a group-level capacity-constrained assignment between attribute groups and vector regions, reducing the combinatorial search space. Second, we construct deterministic surrogates that precisely map primary keys to attribute values and vector regions, enabling in-memory computation of query hardness for arbitrary parameters rather than costly query execution. The deviation from the target hardness then feeds a heuristic search that iteratively adjusts the parameters until the computed hardness matches the target. Extensive experiments demonstrate its superiority in accuracy, efficiency, scalability, and utility.**

***Index Terms*—FANNS, data generation, workload generation**

I. INTRODUCTION

Nearest neighbor search (NNS) over high-dimensional vector data embedded from various modalities, such as text, image, and video, is widely used, where items are represented as vectors and similarity is measured by the distance between embeddings. Since exact NNS is prohibitively expensive on large-scale datasets, approximate nearest neighbor search (ANNS) is commonly adopted to enable efficient, low-latency retrieval [1], [2]. Moreover, many use cases, including retrieval-augmented generation for large language models (LLMs) [3]–[5], bioinformatics [6], and recommendation systems [7], [8], require retrieving items that satisfy attribute-based filtering conditions, such as product categories and price ranges. These requirements have driven the rapid development of filtered approximate nearest neighbor search (FANNS) [9]–[12].

Although several benchmarks have been proposed for FANNS evaluation, they still have two key limitations (see Table I). First, they primarily rely on fixed, publicly available datasets and lack explicit control over *attribute–vector dependencies*. Such dependencies characterize how attribute values are distributed across vector space. They affect the spatial distribution of filtered tuples and thus the performance of FANNS algorithms [13]. Without such control, users cannot systematically evaluate FANNS algorithms under different attribute–vector dependencies. Second, they generally rely on predefined query templates and parameters, lacking flexibility to generate diverse queries under customizable constraints, such as complex predicates, target selectivities, and query–filter correlations [14]. Even on the same dataset, different filter predicates and query vectors can exhibit different selectivities and query–filter correlations, leading to varying query hardness. Controlling these properties is therefore essential for evaluating FANNS performance across different query scenarios. These limitations hinder systematic benchmarking and performance tuning in industrial scenarios, where diverse data and query workloads are needed but real-world workloads may be unavailable due to privacy concerns [15], [16]. They motivate a customizable framework that jointly generates attribute–vector datasets and FANNS workloads under user-specified constraints. However, achieving this goal presents several challenges.

TABLE I
COMPARISON OF FANNS BENCHMARKS.

Category	Capability	BVB [4]	Iff et al. [17]	VecBench [18]	Lu et al. [19]	GRIFFIN
Data	Customizable data	×	×	×	×	✓
	Attr.–vector dep.	×	×	×	×	✓
Query	Query selectivity	×	×	✓	✓	✓
	Query–filter corr.	×	×	×	×	✓
	Arbitrary templates	×	×	×	×	✓

C1: Attribute–vector dependencies hinder scalable joint data generation. Comprehensive FANNS evaluation requires attribute–vector datasets whose characteristics can be controlled on demand and at scale [17]. These characteristics involve two types of constraints: (i) attribute- and vector-side constraints, such as the distributions of relational attributes and the cluster structure of the vector space, and (ii) attribute–vector constraints that specify the dependencies between them. Satisfying the attribute- and vector-side constraints is relatively straightforward because attributes and vectors can be generated independently using established data synthesis techniques [20]. In contrast, achieving fine-grained attribute–vector dependency control without sacrificing generation scalability remains difficult [18]. A naive strategy generates attributes and vectors independently and then searches over their tuple-level alignments by repeatedly permuting attribute values or vectors until the measured attribute–vector dependency approaches a target value. However, the space of possible attribute–vector alignments grows factorially with the dataset size. For a dataset with N tuples, the two independently generated attributes

and vectors induce $N!$ possible tuple-level alignments, which renders exhaustive search computationally prohibitive.

C2: Query hardness is difficult to control due to combinatorial parameter search space and costly validation. Comprehensive FANNS benchmarking requires diverse query workloads with different hardness targets, which are essential for revealing system performance bottlenecks and facilitating performance diagnosis [2]. For a given query template, FANNS query hardness is primarily determined by two factors [18], [21]: 1) the *selectivity* of relational filters, i.e., the fraction of data items that satisfy the filter conditions, and 2) the *query–filter correlation*, i.e., the degree to which filtered attribute values are correlated with the vector distribution. Given a query template, we aim to instantiate both predicate parameters and the query vector to satisfy user-specified *selectivity* and *query–filter correlation* targets. To address this problem, existing work like VecBench [18] first randomly samples a query vector, examines the attribute frequencies among its neighboring vectors, and then selects filtering values to approximate a target selectivity. However, the achievable *selectivity* and *query–filter correlation* depend on the sampled neighborhood and could deviate from the specified targets. More fundamentally, supporting arbitrary query templates while accurately satisfying hardness targets introduces substantial search and validation costs. The reason is that multiple predicate parameters jointly determine the filtered tuples, yielding a combinatorial parameter search space, and the naive trial-and-error strategy is prohibitively expensive, since each validation may require data scans, selectivity computation, and vector similarity computation. As a result, hardness-targeted query generation remains challenging.

To address these challenges, we propose GRIFFIN, a customizable framework that jointly generates attribute–vector datasets and FANNS query workloads satisfying user-specified attribute–vector data dependencies and query hardness requirements. To address **C1**, GRIFFIN reformulates attribute–vector dependency control from tuple-level alignment into a group-level capacity-constrained assignment problem between relational attribute groups and vector regions. Rather than exploring the $N!$ possible tuple-level alignments, it only decides how tuples of each attribute group are placed in each vector region. This reduces the search space from the entire data space to a mapping space whose size depends only on the number of attribute groups and vector regions, independent of the size of the dataset. GRIFFIN then employs a coarse-to-fine search strategy to efficiently find an assignment that satisfies the target dependency. To address **C2**, GRIFFIN constructs deterministic surrogates that exactly map primary keys to attribute values and vector regions. These surrogates enable lightweight computation of query hardness under given parameters through in-memory computation, substantially reducing the expensive scanning and validation overhead of trial-and-error strategies that rely on actual query execution. Finally, GRIFFIN employs an effective heuristic search algorithm to fill in predicate parameters so as to meet given hardness targets. The search is guided by the deviation between the surrogate-computed hardness and the target, which reliably steers the parameter-filling process toward the target hardness instead of relying on random enumeration.

In summary, we make the following contributions.

- We propose GRIFFIN [22], a customizable framework that jointly generates attribute–vector datasets and FANNS query workloads satisfying user-specified attribute–vector dependencies and hardness requirements.
- We reformulate attribute–vector dependency control as a group-level assignment problem to generate data efficiently, and introduce a surrogate-guided query generation algorithm to instantiate parameters with lightweight computation.
- We conduct extensive experiments to demonstrate GRIFFIN's generation accuracy, efficiency, scalability, and utility.

II. Related Work

Customizable query workload generation. SQLSmith [23] randomly generates syntactically valid queries while allowing configurations like the number of joined tables. LearnedSQLGen [24] further employs reinforcement learning to generate queries satisfying a target cost. More recently, SQLBarber [25] and SQLStorm [26] leverage large language models to generate workloads from requirements expressed in natural language. However, these methods cannot generate FANNS queries that jointly satisfy target selectivity and query–filter correlation.

Constraint-aware data generation. Mirage [27] formulates query cardinality requirements as constraints and employs constraint programming to generate data satisfying these constraints. SAM [15] learns a mapping from query cardinalities to data distributions using a machine learning model. DBRepro [28] incorporates global database statistics into constraint-based generation to preserve the original data distribution while satisfying target query cardinalities. However, these methods do not support explicit control over attribute–vector dependencies.

FANNS benchmarking. BigVectorBench (BVB) [4], Iff et al. [17], and Zhu et al. [29] evaluate FANNS using existing datasets with manually specified or randomly generated queries, but do not support customizable data or query generation. Lu et al. [19] and VecBench [18] construct workloads by first sampling query vectors and then determining qualifying data based on their neighboring vectors, producing queries with different selectivities and implicitly determined correlations. However, they provide limited support for complex predicates and cannot generate queries with user-specified hardness.

III. Background

A. Preliminaries

Filtered approximate nearest neighbor search. An attribute–vector dataset can be modeled as a table $T(\mathsf{pk}, A_1, \ldots, A_m, V_1, \ldots, V_r)$, where pk is the primary key, each attribute A_i stores a scalar value, and each vector field V_j stores a d_j-dimensional vector. Given an attribute–vector dataset $\mathcal{D}$, a FANNS query is denoted as $Q{=}(\mathbf{q}, F, K)$ [21]. Here, $\mathbf{q} \in \mathbb{R}^{d_j}$ is the query vector over a vector field V_j, F is a filter predicate over relational attributes, and K is the number of nearest neighbors to retrieve. The predicate F can be generally represented in disjunctive normal form [30]: $F{=}\bigvee_{x=1}^{n} \mathit{clause}_x$, where $\mathit{clause}_x{=}\bigwedge_{y=1}^{m_x} \mathit{literal}_{xy}$ and each literal is $\mathrm{Exp}_{xy} \odot \theta_{xy}$,

with Exp_{xy} an expression over one or more relational attributes, θ_{xy} a constant value, and $\odot \in \{=, \neq, >, \geq, <, \leq\}$ (including `(not)in` and `between and`). The goal of FANNS is to retrieve the top-K objects with the smallest distances among those satisfying F.

Attribute–vector data dependency (ρ). Real-world vectors commonly exhibit clustered structures [31], and nearby vectors are often accessed together in FANNS; therefore, the embedding space can usually be divided into vector regions $\mathcal{R}$. Let $Z(o) \in \{1, \ldots, |\mathcal{R}|\}$ denote the region containing the embedding of object o, where Z is a discrete random variable over the region identifiers. Because a vector embedding is high-dimensional and continuous, directly measuring the dependency between an attribute and a vector field is difficult. Since the embedding space can be divided into regions, the dependence between an attribute and the vector field can instead be measured as the dependence between the attribute and the vector region indicator, for which metrics are well established: the correlation ratio for a continuous attribute and normalized mutual information for a discrete one [32]–[34]. We therefore use the region Z as a discrete proxy of the vector field and quantify the attribute–vector dependency by the variation of the attribute distribution across vector regions. Formally, for an attribute A and a vector field V whose region proxy is Z, we define the dependency strength ρ as the normalized statistical dependence between A and Z. Specifically, for an attribute A_i and a vector field V_j, we denote their dependency strength by ρ_{ij} and its target value by ρ^*_{ij}.

- *For a continuous attribute* A, we adopt the correlation ratio (fraction of explained variance), a classical measure of association between a continuous variable and a discrete grouping [32]: $\rho = \frac{\mathrm{Var}(\mathbb{E}[A|Z])}{\mathrm{Var}(A)} \in [0, 1]$. Here, $\mathbb{E}[A \mid Z]$ denotes the mean value of A within each vector region. The numerator measures the variance of the regional means, while the denominator normalizes it by the overall variance of A. Thus, ρ represents the proportion of variation in A that is associated with the partition of the vector regions [35].
- *For a discrete attribute* A, we adopt normalized mutual information, a classical measure of dependence between two discrete variables [36], [37]: $\rho = \frac{I(A;Z)}{H(A)} \in [0, 1]$. Here, $I(A; Z) = \sum_{a,z} p(a, z) \log \frac{p(a,z)}{p(a)p(z)}$ denotes the mutual information between attribute values and vector regions, and $H(A) = -\sum_a p(a) \log p(a)$ is the entropy of A. Intuitively, $I(A; Z)$ measures how much knowing the vector region reduces the uncertainty about the attribute value, and $H(A)$ normalizes it by the total uncertainty; ρ is thus the proportion of attribute uncertainty explained by the region partition.

Query–filter correlation (GLS). The *query–filter correlation* is generally measured by the Global-Local Selectivity (GLS) metric [13]. The global selectivity is the fraction of objects satisfying F: $\sigma_g(F) = \frac{|\{o_i \in \mathcal{D} | F(o_i) = \text{true}\}|}{|\mathcal{D}|} \in [0, 1]$. Furthermore, let $\mathcal{N}_K(\mathbf{q})$ denote the exact unfiltered K nearest neighbors of the query vector $\mathbf{q}$. The local selectivity is the fraction of the K nearest neighbors around $\mathbf{q}$ satisfying F: $\sigma_l(\mathbf{q}, F, K) = \frac{|\{o_i \in \mathcal{N}_K(\mathbf{q}) | F(o_i) = \text{true}\}|}{K} \in [0, 1]$. Normalizing by the global baseline gives $\eta(\mathbf{q}, F, K) = \frac{\sigma_l(\mathbf{q}, F, K)}{\sigma_g(F)} \in [0, \infty)$. $\eta(\mathbf{q}, F, K)$ compares the local occurrence rate of filtered objects around $\mathbf{q}$ with their global rate, thereby characterizing their spatial distribution relative to the query vector. It finally maps this ratio to a bounded score:

$$\mathrm{GLS}(\mathbf{q}, F, K) = \frac{\eta(\mathbf{q}, F, K) - 1}{\eta(\mathbf{q}, F, K) + 1} \in [-1, 1). \tag{1}$$

A positive GLS indicates local alignment between the filter and the vector neighborhood, while a negative GLS indicates misalignment, which may make the FANNS problem harder to solve. Moreover, from the definition of η, the local selectivity for a target GLS is

$$\sigma_l(\mathbf{q}, F, K) = \sigma_g(F) \cdot \frac{1 + \mathrm{GLS}(\mathbf{q}, F, K)}{1 - \mathrm{GLS}(\mathbf{q}, F, K)}. \tag{2}$$

B. Problem Definition

Definition 1 (Customizable FANNS Data and Query Generation)**.** Given a database schema $\mathcal{S}$, the attribute- and vector-side data constraints $\mathcal{C}_d$, the target attribute–vector dependencies $\mathcal{T} = \{(A_i, V_j, \rho^*_{ij})\}$, and w parameterized FANNS query templates $\widehat{\mathcal{Q}} = \{\widehat{Q}_1, \ldots, \widehat{Q}_w\}$ with the query constraints $\mathcal{C}_q$, we aim to generate an attribute–vector dataset $\mathcal{D}$, together with an instantiated query workload set $\mathcal{Q} = \{Q_1, \ldots, Q_w\}$. Specifically, $\mathcal{D}$ should satisfy $\mathcal{S}$ and $\mathcal{C}_d$, while ensuring that the achieved dependency strength ρ_{ij} is close to the target ρ^*_{ij} in $\mathcal{T}$. Similarly, each query $Q_k = (\mathbf{q}_k, F_k, K_k)$ should satisfy the hardness targets specified for $\widehat{Q}_k$ in $\mathcal{C}_q$, i.e., its target *selectivity* and *query–filter correlation (GLS)*.

Consider a dataset with category, price, and image vector v, following a skewed category distribution (60%/30%/10%), uniform prices in $[1, 10]$, and 32 vector clusters, with a target category–vector dependency ρ^*=0.5. Given the query template "`SELECT * FROM` T `WHERE category = ? OR price < ? ORDER BY` v `<-> ? LIMIT` K", GRIFFIN generates the dataset and instantiates query parameters to meet target hardness, e.g., σ^*_g=10% and GLS^* =0.5.

IV. GRIFFIN OVERVIEW

The framework of GRIFFIN is illustrated in Fig. 1. It takes schema $\mathcal{S}$, data constraints $\mathcal{C}_d$ and $\mathcal{T}$, query templates $\widehat{\mathcal{Q}}$, and query constraints $\mathcal{C}_q$ as inputs.

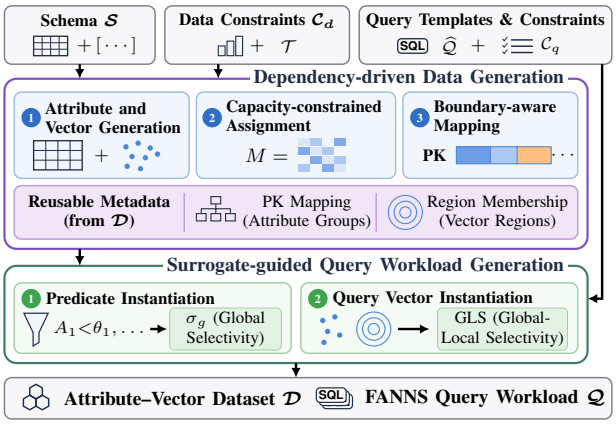


Fig. 1. GRIFFIN's framework.

Dependency-driven data generation. This component generates attribute–vector data satisfying $\mathcal{C}_d$ and $\mathcal{T}$. Existing techniques support generating attributes and vectors independently [20]. However, to satisfy the target attribute–vector dependencies, they swap values or vectors across tuples, which gives rise to $N!$ possible tuple-level alignments and makes exhaustive search prohibitively expensive. Moreover, even if such swapping reaches an alignment that meets the target attribute–vector dependencies, it fails to model these dependencies and thus yields no reusable structure to guide subsequent query workload generation under the query constraints. So the generation process is still forced to rely on expensive data scans and vector searches. To address these challenges, GRIFFIN designs a dependency-driven data generation strategy which builds reusable mapping metadata to serve as deterministic surrogates. It can exactly map primary keys to attribute values and vector regions. ❶ GRIFFIN first independently generates relational attributes and vectors according to their respective constraints, organizing them into *attribute groups* of the same attribute values and *vector regions* of spatially close vectors, and assigns primary keys (PKs) contiguously within each group and within each region. ❷ It then formulates attribute–vector dependency control as a group-level capacity-constrained assignment problem using an *assignment matrix* whose size depends only on the numbers of attribute groups and vector regions. Each entry specifies the number of tuples assigned to a pair of attribute-group and vector-region. To reach a target attribute–vector dependency, GRIFFIN follows a coarse-to-fine search strategy: it builds two anchor matrices for weak and strong dependencies, interpolates between them to approximate the target, and applies fine-grained refinement to further reduce the error. ❸ Finally, since each generated tuple contains both relational attributes and a vector, GRIFFIN must assign a unique PK from which both its attribute group and vector region can be decoded. Because the two partitions cross-cut each other, no single PK ordering can keep both attribute groups and vector regions contiguous. GRIFFIN therefore keeps each vector region with the initialized contiguous PK range and applies Boundary-aware Mapping to recover the attribute group of a PK. Specifically, within each region, the PK range is split into consecutive segments, one per attribute group, whose lengths equal the corresponding entries of the assignment matrix, i.e., the number of tuples of that group in the current region. Given a PK, GRIFFIN first locates its vector region, then finds its attribute group via the segment boundaries (e.g., binary search).

Surrogate-guided query workload generation. The component generates FANNS workloads satisfying $\mathcal{C}_q$. Each query involves two types of parameters, i.e., predicate parameters and a query vector. Trial-and-error approaches that repeatedly sample these parameters and execute queries until hardness targets are satisfied are prohibitively expensive due to the large search space and the high cost of validating candidates through actual query execution. To address these challenges, GRIFFIN adopts a surrogate-based strategy that replaces trial-and-error query execution and builds on the surrogates established during data generation. Since GLS depends on the spatial distribution of tuples satisfying the filter predicate, predicate instantiation precedes query vector instantiation. ❶ For *Filter Predicate Instantiation*, GRIFFIN computes a candidate predicate's exact global selectivity directly from these surrogates, without executing it, and employs a genetic algorithm (GA) over these evaluations to search for predicate parameters that meet the target selectivity. ❷ For *Query Vector Instantiation*, GRIFFIN aims to identify a query vector satisfying the target GLS. Given the known global selectivity of F, the target GLS can be translated into a local selectivity, defined as the fraction of tuples satisfying F among the top-K of a query vector. The problem thus reduces to finding a query vector whose local selectivity approaches this target. Since a query vector's nearest neighbors tend to lie within the same vector region, and attributes are randomly assigned to vectors within each region, tuples satisfying F are approximately uniformly distributed among the vectors in that region. Consequently, the region's *filter density*, defined as the fraction of its tuples satisfying F, provides an estimate of the local selectivity of query vectors sampled from that region. The surrogates map each PK to its attribute value and vector region, so these densities are counted easily. GRIFFIN therefore samples candidate vectors from regions whose densities are close to the target local selectivity, evaluates their exact GLS through top-K searches, and applies simulated annealing (SA) to refine the best candidate until the target GLS is satisfied.

V. ATTRIBUTE–VECTOR DATA GENERATION

Generating attribute–vector datasets with controlled dependencies via naive tuple-level assignment is too expensive. To address this, GRIFFIN organizes attributes and vectors into *attribute groups* and *vector regions*, respectively, and materializes PK-to-attribute and PK-to-region mappings as **deterministic surrogates** (§ V-A). It then reformulates dependency control as a **group-level capacity-constrained assignment** problem, solved by an efficient **coarse-to-fine search** strategy to meet the target dependencies (§ V-B).

A. Attribute and Vector Generation

Vector generation and region organization. Note that each tuple in an attribute–vector dataset is uniquely identified by a PK. Given a PK, the database can quickly locate the corresponding tuple and retrieve its attributes and associated vectors. Conversely, if arbitrary attribute values can be mapped to their corresponding PKs, we can derive both the query *selectivity*, by counting the number of filtered tuples identified by unique PKs, and *GLS*, by the vector spatial distribution of the filtered tuples around the query vector. Motivated by this, GRIFFIN proposes to construct tractable mappings between PKs and the generated relational attributes and vectors while preserving their respective constraints. GRIFFIN first randomly generates a set of center vectors, whose number is specified by the user [38]. It then determines the number of vectors assigned to each center according to the user-specified cluster size distribution and generates vectors around each center with the specified variance. Note that these *generation*

clusters only govern the statistical properties of the generated vectors. They do not necessarily correspond to the actual spatial partitions of the vectors, because vectors generated from different centers may substantially overlap. GRIFFIN therefore applies KMeans [39] to partition the generated vectors into *vector regions* $\mathcal{R}=\{R_1,\ldots,R_{|\mathcal{R}|}\}$, each grouping spatially close vectors and compactly representing their locality [40]. Each generated vector is assigned a PK as its identifier. However, randomly assigning PKs can scatter vectors from the same region across different PK ranges, requiring an explicit PK-to-region mapping; GRIFFIN instead assigns consecutive PKs within each region so that all vectors in a region share a contiguous PK interval; since PKs are only identifiers, this assignment does not alter their spatial relationships, and the mapping reduces to the interval boundaries. Let $|R_i|$ be the number of vectors in R_i. With $B_i=\sum_{t=1}^{i}|R_t|$ and $B_0=0$, region R_i covers the contiguous PK interval $\mathcal{I}_i^{\text{vec}}=[B_{i-1},B_i)$, so GRIFFIN only stores the ordered boundary list $\mathcal{B}^{\text{vec}}=[B_1,\ldots,B_{|\mathcal{R}|}]$. Given a PK, its vector region can be determined by locating the interval in $\mathcal{B}^{\text{vec}}$ that contains the PK. For example, in Fig. 2, cardinalities $(2,3,1)$ yield boundaries $\mathcal{B}^{\text{vec}}=[2,5,6]$ and PK intervals $[0,2),[2,5),[5,6)$ for the three regions.

Relational-attribute generation. After fixing the PK layout, GRIFFIN generates each relational attribute according to its number of distinct values (NDV) and distribution (e.g., Zipfian). For an attribute A, its generated values are organized into $\mathcal{G}=\{G_1,\ldots,G_{|\mathcal{G}|}\}$ with $|\mathcal{G}|=\text{NDV}(A)$; each group holds one distinct value, and its cardinality $|G_j|$ follows the target distribution (e.g., equal-sized for uniform). Similarly, with $P_j=\sum_{t=1}^{j}|G_t|$ and $P_0=0$, group G_j covers the PK interval $\mathcal{I}_j^{\text{attr}}=[P_{j-1},P_j)$, and GRIFFIN stores the ordered group-boundary list $\mathcal{B}^{\text{attr}}=[P_1,\ldots,P_{|\mathcal{G}|}]$. We then use a deterministic mapping function to convert each group identifier into a concrete attribute value. For a continuous attribute, this mapping is expressed as $A=f_A(j)$, where f_A is defined over the group identifiers. For a discrete attribute, GRIFFIN maintains a dictionary that maps each group identifier to a category label. After these processes, GRIFFIN retains three types of metadata, i.e., $\mathcal{B}^{\text{vec}}$, $\mathcal{B}^{\text{attr}}$, and f_A for each attribute. Together, they serve as deterministic surrogates that exactly map primary keys to attribute values and vector regions.

B. Attribute–Vector Dependency Control

After independently generating the attributes and vectors, the remaining task is to establish the target attribute–vector dependency between them. Since attribute–vector dependency reflects the distribution of attribute-group occurrences across vector regions, GRIFFIN controls it by assigning group occurrences to regions while satisfying capacity constraints: each group contributes a fixed number of tuples, and each region accommodates a fixed number of tuples. The assignment is then optimized to achieve the target dependency. GRIFFIN represents such an assignment using an *assignment matrix*. Given the vector regions $\mathcal{R}=\{R_1,\ldots,R_{|\mathcal{R}|}\}$ and the attribute groups $\mathcal{G}=\{G_1,\ldots,G_{|\mathcal{G}|}\}$, it constructs a nonnegative integer matrix $M=\begin{bmatrix} M_{11} & \cdots & M_{1|\mathcal{G}|} \\ \vdots & \ddots & \vdots \\ M_{|\mathcal{R}|1} & \cdots & M_{|\mathcal{R}||\mathcal{G}|} \end{bmatrix}\in\mathbb{Z}_{\geq 0}^{|\mathcal{R}|\times|\mathcal{G}|}$, where each entry M_{ij} denotes the number of tuples whose vectors belong to region R_i and whose attribute values belong to group G_j. Let r_i and c_j denote the cardinalities of region R_i and group G_j (with N the total number of tuples), and $\mathbf{r}=(r_1,\ldots,r_{|\mathcal{R}|})$, $\mathbf{c}=(c_1,\ldots,c_{|\mathcal{G}|})$ the row- and column-sum vectors. The individual entries of M then determine how attribute-group occurrences are distributed across vector regions. Therefore, given a user-specified target dependency ρ^*, GRIFFIN has the following optimization goal:

$$\begin{aligned} \min_{M} \quad & |\rho(M)-\rho^*| \\ \text{s.t.} \quad & \sum_j M_{ij}=r_i,\ \sum_i M_{ij}=c_j,\ M_{ij}\in\mathbb{Z}_{\geq 0}. \end{aligned} \tag{3}$$

Here, $\rho(M)$ is the value of ρ induced by the assignment M, which specifies the joint distribution of attribute groups and vector regions. Directly solving this problem is challenging for two reasons: the number of feasible assignments grows combinatorially, making exhaustive search prohibitively expensive; and $\rho(M)$ is a nonlinear function, which standard linear optimization solvers (e.g., OR-Tools) cannot directly handle [41]. Fortunately, the semantics of ρ provide useful guidance for constructing an assignment matrix with the target dependency. A **weak dependency** indicates little variation in attribute statistics across regions, with similar regional means for continuous attributes and similar distributions for discrete attributes. In contrast, a **strong dependency** exhibits greater variation by concentrating attribute groups in fewer regions. These patterns define two reference assignments: the **low-dependency anchor** M^{low} representing statistical independence and the **high-dependency anchor** M^{high} representing a concentrated assignment. More importantly, these anchors exhibit two properties that enable an efficient search strategy to accurately achieve a specified target dependency strength, which we call *anchor-based search*. First, any interpolation between the two anchors, as defined in Eq. 4, preserves the prescribed marginal distributions of attribute groups and vector regions (Lemma 1). Second, the dependency strength ρ changes monotonically as α increases from 0 to 1 (Theorem 1). This property provides a direct guide for α search. That is, increasing α strengthens the dependency, while decreasing α weakens it.

$$\widetilde{M}(\alpha)=(1-\alpha)M^{\text{low}}+\alpha M^{\text{high}},\qquad \alpha\in[0,1]. \tag{4}$$

Together, these two properties successfully reduce the search for a target dependency from a high-dimensional space of assignment matrices to a one-dimensional search over the interpolation coefficient $\alpha\in[0,1]$. To further reduce the search cost, GRIFFIN adopts a *coarse-to-fine* strategy (Alg. 1) that first performs a global search along the interpolation path to approximate the target dependency, then converts the resulting matrix into an integer assignment before locally refining it. Finally, since the resulting assignment matrix changes how attribute groups are distributed across vector regions, GRIFFIN employs a *Boundary-aware Mapping* strategy to reconstruct the PK-to-attribute-value relationships.

Coarse-search stage. GRIFFIN first constructs two anchor matrices (Line 1). For the low-dependency anchor, each

Algorithm 1: Attribute–Vector Dependency Control

Input: Region cardinalities $\mathbf{r}$; group cardinalities $\mathbf{c}$; target dependency ρ^*; search and refinement budget T_b, T_r;
Output: Assignment matrix M;

1 $M^{\text{low}} \leftarrow$ BUILDLOWDEPENDENCYANCHOR$(\mathbf{r}, \mathbf{c})$;
  $M^{\text{high}} \leftarrow$ BUILDHIGHDEPENDENCYANCHOR$(\mathbf{r}, \mathbf{c})$;
2 $(\alpha^*, \widetilde{M}(\alpha^*)) \leftarrow$ BINARYSEARCH$\big(M^{\text{low}}, M^{\text{high}}, \rho^*, T_b\big)$;
3 $M \leftarrow$ INTEGERIZE$\big(\widetilde{M}(\alpha^*), \mathbf{r}, \mathbf{c}\big)$; $\varepsilon \leftarrow |\rho(M) - \rho^*|$;
4 **for** $t \leftarrow 1$ **to** T_r **do**
5 $\Delta^* \leftarrow$ BESTFEASIBLECYCLEMOVE(M, ρ^*);
6 **if** $\Delta^* = \varnothing$ **then break**;
7 $M' \leftarrow$ APPLYCYCLEMOVE(M, Δ^*);
8 $\varepsilon' \leftarrow |\rho(M') - \rho^*|$;
9 **if** $\varepsilon' \geq \varepsilon$ **then break**;
10 $M \leftarrow M'$; $\varepsilon \leftarrow \varepsilon'$;
11 **return** M;

attribute group is distributed proportionally across all regions, with $M^{\text{low}}_{ij} = r_i \cdot \frac{c_j}{N}$. Each vector region thus contains attribute groups in the same proportions as in the entire dataset, resulting in nearly zero dependency. For the high-dependency anchor M^{high}, GRIFFIN adopts different assignment strategies for continuous and discrete attributes, respectively. For a continuous attribute, it sorts tuples by value and partitions them into contiguous segments, assigning each segment to a distinct region to increase the variance of regional means. For a discrete attribute, it greedily assigns tuples with the same value to the same region, splitting them only when required by region capacities. For example, in Fig. 2, G_1 and G_2 are assigned to R_1 and R_2, respectively, while G_3 is split between R_2 and R_3 due to capacity constraints. Both strategies thus reduce the conditional entropy of attribute values given regions and strengthen the dependency. Based on the two anchors, GRIFFIN performs binary search over $\alpha \in [0, 1]$ to get α^* and approximate the target ρ^* (Line 2), leveraging the monotonicity of dependency with respect to α. The resulting assignment $\widetilde{M}(\alpha^*)$ preserves the marginals $\mathbf{r}$ and $\mathbf{c}$ but may contain fractional entries. GRIFFIN therefore integerizes $\widetilde{M}(\alpha^*)$ while preserving these constraints (Line 3), then refines M to further reduce the dependency error.

Refinement stage. Since the dependency is already close to the target after the coarse-search stage, only minor adjustments are needed. However, changing a single entry of M would violate both its row and column constraints: an increase in one entry must be accompanied by a decrease in another entry of the same row, together with corresponding adjustments in the same columns. The smallest structure supporting such an adjustment is a 2×2 submatrix formed by two vector regions and two attribute groups, so GRIFFIN employs 2×2 cycle moves for local refinement (Lines 4–10). Given two vector regions R_{i_1} and R_{i_2} and two attribute groups G_{j_1} and G_{j_2}, a cycle move updates the corresponding 2×2 submatrix by $\Delta M = \delta \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$, or in the reverse direction, where δ is chosen so that all updated entries remain nonnegative. Moreover, since M is an integer matrix, $\delta=1$ gives the smallest nonzero adjustment. GRIFFIN therefore uses $\delta=1$ by default

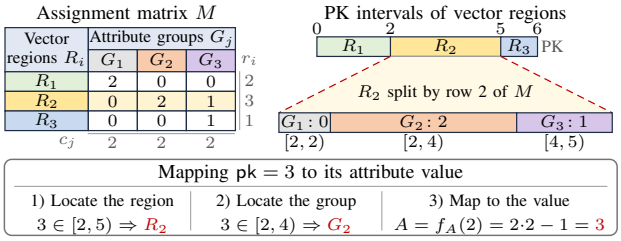


Fig. 2. Example of PK remapping.

for fine-grained refinement, allowing the dependency to be adjusted toward the target with minimal changes to the current assignment. If $\rho(M) < \rho^*$, it concentrates group occurrences into fewer regions to strengthen the dependency; otherwise it distributes them more evenly to weaken it. At each iteration, GRIFFIN randomly samples a fixed number of feasible cycle moves with $\delta=1$ and selects the move Δ^* that minimizes the resulting dependency error ε'. The selected move is applied only if it reduces the current error. The refinement terminates when no feasible move is found (Line 6), when the best sampled move fails to reduce the error (Line 9), or when the maximum number of refinement iterations T_r is reached (Line 4). Although each cycle move modifies only four entries, a sequence of such moves can still cover the entire feasible space (Theorem 2).

PK remapping. In § V-A, PKs were assigned to groups and regions independently, so a PK's group and region are directly identifiable from these surrogates. However, M may assign tuples of the same group to different regions, so the original PK-to-attribute-group mapping must be reassigned. Explicitly storing the assignments would require an additional $O(N)$ storage cost. Instead, since PKs within each region form a consecutive interval and each row of M specifies how many PKs in that interval are assigned to each attribute group, GRIFFIN divides the PK interval of region R_i into consecutive segments of lengths $M_{i1}, M_{i2}, \ldots, M_{i|\mathcal{G}|}$, where the j-th segment corresponds to attribute group G_j. Based on this structure, GRIFFIN constructs a *Boundary-aware Mapping* through three steps by using the vector-region boundaries $\mathcal{B}^{\text{vec}}$, the attribute-group boundaries $\mathcal{B}^{\text{attr}}$, and the assignment matrix M: 1) *Locate the region:* Given a PK, identify its vector region R_i using the region boundaries $\mathcal{B}^{\text{vec}}$; 2) *Locate the group:* Determine the corresponding attribute group G_j using the PK's offset within R_i and the cumulative entries of row M_i; 3) *Map to the value:* Use the attribute mapping function f_A to obtain the attribute value associated with the identified group G_j. Thus, the PK-to-attribute-value mapping can be reconstructed through $\mathsf{pk} \to R_i \to G_j \to A$.

For example, consider six tuples with region cardinalities $(2, 3, 1)$, i.e., R_1=$[0, 2)$, R_2=$[2, 5)$, R_3=$[5, 6)$, and group cardinalities $(2, 2, 2)$. As shown in Fig. 2, the second row $[0, 2, 1]$ of the assignment matrix M means that no tuples in R_2 are assigned to G_1, two to G_2, and one to G_3, partitioning R_2 into $[2, 2), [2, 4), [4, 5)$ corresponding to $[G_1, G_2, G_3]$. For pk=3 $\in R_2$, its offset within R_2 falls into the second subinterval, so pk=3 is assigned to G_2; with the mapping function $f_A(j)$=$2 \cdot j - 1$, the attribute value is A=$f_A(2)$=3.

Lemma 1. *For any $\alpha \in [0, 1]$, $\widetilde{M}(\alpha)$ is nonnegative and*

satisfies $\sum_j \widetilde{M}_{ij}(\alpha)=r_i$, *and* $\sum_i \widetilde{M}_{ij}(\alpha)=c_j$. *Thus,* $\widetilde{M}(\alpha)$ *preserves the prescribed marginal distributions over vector regions and attribute groups.*

Proof. The two anchors are nonnegative and share the same row sums r_i and column sums c_j; the latter equal the region and group cardinalities. For $\alpha \in [0,1]$, each entry $\widetilde{M}_{ij}(\alpha)$ is a convex combination of two nonnegative numbers and is therefore nonnegative. Summing along a row and along a column gives

$$\begin{aligned}\sum_j \widetilde{M}_{ij}(\alpha) &= (1-\alpha)\sum_j M^{\text{low}}_{ij} + \alpha \sum_j M^{\text{high}}_{ij} \\ &= (1-\alpha)r_i + \alpha r_i = r_i, \\ \sum_i \widetilde{M}_{ij}(\alpha) &= (1-\alpha)\sum_i M^{\text{low}}_{ij} + \alpha \sum_i M^{\text{high}}_{ij} \\ &= (1-\alpha)c_j + \alpha c_j = c_j.\end{aligned}$$

Normalizing by the common total $N=\sum_i r_i=\sum_j c_j > 0$ turns these sums into the marginal probabilities r_i/N and c_j/N, which are independent of α; hence both marginals are preserved along the path. □

Theorem 1. *Suppose* M^{low} *represents statistical independence between the attribute group and vector region, and* M^{high} *has the same row and column sums. For* $\widetilde{M}(\alpha)=(1-\alpha)M^{\text{low}} + \alpha M^{\text{high}}$, $\alpha \in [0,1]$, $\rho(\widetilde{M}(\alpha))$ *is monotonically nondecreasing with* α *for both continuous and discrete attributes, assuming* $\mathrm{Var}(A) > 0$ *and* $H(A) > 0$, *respectively.*

Proof. Let $N=\sum_i r_i=\sum_j c_j$ be the number of tuples, $p_i=r_i/N$, and $q_j=c_j/N$; empty regions and groups are excluded, so $p_i > 0$ and $q_j > 0$. Dividing $\widetilde{M}(\alpha)$ by N gives the joint distribution of the attribute A and the region indicator Z at interpolation level α. By Lemma 1 the marginals are p_i and q_j for every α, so the marginal distribution of A does not depend on α. Consequently $\mathrm{Var}(A)$ and $H(A)$ are fixed, and both scores are monotone exactly when their numerators $\mathrm{Var}(\mathbb{E}[A \mid Z;\alpha])$ and $I_\alpha(A;Z)$ are.

Continuous attribute. Every tuple in group G_j carries the same value $a_j=f_A(j)$ (see § V-A), so the mean of A in region R_i is

$$\mu_i(\alpha)=\frac{1}{r_i}\sum_j \widetilde{M}_{ij}(\alpha)a_j=(1-\alpha)\mu_i^{\text{low}} + \alpha\mu_i^{\text{high}},$$

where μ_i^{low} and μ_i^{high} are the corresponding means under the two anchors. Because the anchors share the marginals, the same r_i divides both, so the conditional mean is a convex combination with the same α as the matrix. Under M^{low} the attribute is independent of the region, hence $\mu_i^{\text{low}}=\mu=\mathbb{E}[A]$ for all i, and

$$\mu_i(\alpha) - \mu = \alpha\big(\mu_i^{\text{high}} - \mu\big).$$

By the law of total variance,

$$\mathrm{Var}(\mathbb{E}[A \mid Z;\alpha])=\sum_i p_i\big(\mu_i(\alpha)-\mu\big)^2=\alpha^2\sum_i p_i\big(\mu_i^{\text{high}}-\mu\big)^2. \tag{5}$$

The sum on the right is $\mathrm{Var}(\mathbb{E}[A \mid Z])$ evaluated at M^{high}, since p_i and μ are anchor-independent. Dividing by the fixed positive $\mathrm{Var}(A)$ therefore yields

$$\rho_{\text{con}}\big(\widetilde{M}(\alpha)\big)=\alpha^2 \cdot \rho_{\text{con}}\big(M^{\text{high}}\big).$$

As α grows from 0 to 1, α^2 is nondecreasing, so ρ_{con} is nondecreasing on $[0,1]$.

Discrete attribute. Let P^{high} denote the joint distribution of A and Z induced by M^{high}, and let $Q(a,z)=q_A(a)q_Z(z)$ be the product distribution with the same marginals; Q is the joint distribution induced by M^{low}. Interpolation acts entrywise on these frequencies, so the joint distribution at α is

$$P_\alpha=(1-\alpha)Q + \alpha P^{\text{high}}.$$

The marginals of P_α equal those of Q, hence $p_\alpha(a)p_\alpha(z)=Q(a,z)$ for every α, and the mutual information can be written as a divergence from the fixed distribution Q:

$$I_\alpha(A;Z)=\sum_{a,z} P_\alpha(a,z)\log\frac{P_\alpha(a,z)}{Q(a,z)}=D_{\text{KL}}(P_\alpha\|Q),$$

where cells with $P_\alpha(a,z)=0$ contribute 0 by the convention $0\log 0=0$; such cells also have $Q(a,z)=0$, since Q carries the same marginals, so the logarithm is applied only to positive cells. For $0 \le \alpha_1 < \alpha_2 \le 1$, set $\lambda=\alpha_1/\alpha_2 \in [0,1)$. Expanding the two definitions gives

$$\begin{aligned}P_{\alpha_1} &= (1-\alpha_1)Q + \alpha_1 P^{\text{high}} \\ &= (1-\lambda)Q + \lambda\big((1-\alpha_2)Q + \alpha_2 P^{\text{high}}\big) \\ &= (1-\lambda)Q + \lambda P_{\alpha_2},\end{aligned}$$

so P_{α_1} is a convex combination of Q and P_{α_2}. Convexity of $D_{\text{KL}}(\cdot\|Q)$ in its first argument gives

$$\begin{aligned}D_{\text{KL}}(P_{\alpha_1}\|Q) &\le (1-\lambda)\,D_{\text{KL}}(Q\|Q) + \lambda\,D_{\text{KL}}(P_{\alpha_2}\|Q) \\ &= \lambda\,D_{\text{KL}}(P_{\alpha_2}\|Q) \\ &\le D_{\text{KL}}(P_{\alpha_2}\|Q),\end{aligned}$$

where the first term vanishes because $D_{\text{KL}}(Q\|Q)=0$ and the last step uses $\lambda \le 1$ and the nonnegativity of the divergence. Hence $I_\alpha(A;Z)$ is nondecreasing in α, and dividing by the fixed positive $H(A)$ proves the discrete case. □

Theorem 2 (Completeness of Cycle Moves)**.** *For any two nonnegative integer assignment matrices* M *and* M' *with identical row and column sums, there exists a finite sequence of feasible* 2×2 *cycle moves with* $\delta=1$ *that transforms* M *into* M' *while preserving nonnegativity and marginal distributions.*

Proof. Rows index vector regions and columns index attribute groups. If $|\mathcal{R}|=1$ or $|\mathcal{G}|=1$, the margins determine the matrix uniquely, so assume $|\mathcal{R}|, |\mathcal{G}| \ge 2$. We fix the matrix one row at a time, and show that row i can be matched to its target while rows $1,\dots,i-1$ stay fixed. Suppose rows $1,\dots,i-1$ already equal those of M' but row i differs.

Since $\sum_j M_{ij}=\sum_j M'_{ij}=r_i$ and the two rows differ, some column a satisfies $M_{ia} > M'_{ia}$ and some column b satisfies $M_{ib} < M'_{ib}$; if row i never exceeded its target, the equal row sums would force the two rows to coincide. The column sums

of M and M' agree, and rows $1,\dots,i-1$ already coincide, hence

$$\sum_{k>i}\big(M_{kb}-M'_{kb}\big)=\big(M'_{ib}-M_{ib}\big)+\sum_{k<i}\big(M'_{kb}-M_{kb}\big)$$
$$=M'_{ib}-M_{ib}>0,$$

so at least one region R_k with $k>i$ satisfies $M_{kb}>M'_{kb}$. Both entries we decrease are therefore positive integers, $M_{ia}>M'_{ia}\geq 0$ and $M_{kb}>M'_{kb}\geq 0$, and the following unit move is feasible:

$$M_{ia}\leftarrow M_{ia}-1,\quad M_{ib}\leftarrow M_{ib}+1,$$
$$M_{ka}\leftarrow M_{ka}+1,\quad M_{kb}\leftarrow M_{kb}-1.$$

Every entry stays nonnegative and every marginal is preserved: rows i and k each lose and gain one unit, and columns a and b each gain and lose one unit. Rows other than i and k are untouched, so in particular rows $1,\dots,i-1$ remain fixed.

For the progress measure, take the ℓ_1 distance between row i and the target row M'_i. The entries in columns a and b each move one unit toward the target, and all other entries of the row are unchanged, so the distance decreases by exactly two. It is a nonnegative integer, hence row i becomes equal to M'_i after finitely many such moves. Applying the argument for i=$1,\dots,|\mathcal{R}|-1$ fixes the first $|\mathcal{R}|-1$ rows; the remaining row is then determined by the column sums and therefore also equals the last row of M'. The concatenated moves form the required finite sequence. □

Complexity. Let R=$|\mathcal{R}|$ and G=$|\mathcal{G}|$. Dependency control takes $O(G\log G+T_b\cdot R\cdot G+T_r)$ time and $O(R\cdot G)$ space, where T_b and T_r are the coarse-search and refinement budgets.

Constructing the anchors. The anchors are $R\times G$ matrices, so writing their entries costs $O(R\cdot G)$ each. Building the low anchor distributes each group proportionally, M^{low}_{ij}=$r_i\cdot c_j/N$, which takes $O(R\cdot G)$ arithmetic operations. Building the high anchor orders the G groups by value for a continuous attribute and splits that sequence into R contiguous segments according to the cumulative group counts, while for a discrete attribute it fills the regions greedily with whole groups and splits a group only when a region becomes full. Ordering all groups by value costs $O(G\log G)$, the cumulative counts are formed in $O(G)$, and assigning groups to the R regions costs $O(R+G)$. The two anchors therefore cost $O(G\log G+R\cdot G)$ in total.

Coarse search. Each of the T_b iterations selects a coefficient α, writes the $R\cdot G$ entries of $\widetilde{M}(\alpha)$ from the two anchors, and evaluates ρ on the resulting matrix, so one iteration costs $O(R\cdot G)$ and the stage costs $O(T_b\cdot R\cdot G)$. The binary search reuses no partial results across iterations, since each α changes all entries.

Refinement. A 2×2 cycle move modifies four entries, and we maintain the statistics that ρ depends on so that one move is evaluated in $O(1)$ time. For a continuous attribute, the numerator of ρ is a function of the regional sums s_i=$\sum_j M_{ij}a_j$:

$$\mathrm{Var}(\mathbb{E}[A\mid Z])=\sum_i \frac{s_i^2}{N\cdot r_i}-\mu^2,$$

where μ and r_i are fixed by the marginals. A move on regions R_i,R_k and groups G_a,G_b changes s_i by a_b-a_a and s_k by a_a-a_b, leaving all other sums intact; the two affected terms of the sum are updated in $O(1)$, giving the new numerator and hence the new ρ.

For a discrete attribute, expanding the logarithm separates the marginals from the joint frequencies:

$$I(A;Z)=\frac{1}{N}\sum_{i,j}M_{ij}\log M_{ij}-\frac{1}{N}\sum_i r_i\log r_i$$
$$-\frac{1}{N}\sum_j c_j\log c_j+\log N,$$

where the last three terms depend only on the row and column sums and are therefore fixed. Since $H(A)$ is fixed as well, ρ is an affine function of Φ=$\sum_{i,j}M_{ij}\log M_{ij}$. A move changes exactly the four entries $M_{ia},M_{ib},M_{ka},M_{kb}$, so updating four terms of Φ costs $O(1)$ with the convention $0\log 0$=0. Initializing all s_i or Φ costs $O(R\cdot G)$. Each iteration samples a fixed number of feasible moves and evaluates each in $O(1)$, so T_r iterations cost $O(T_r)$; the refinement terminates earlier when no feasible move remains or when the best sampled move no longer reduces the error.

Integerization. Flooring the $R\cdot G$ entries costs $O(R\cdot G)$; the row and column deficits are computed in $O(R+G)$; the fractional remainders are sorted in $O(R\cdot G\cdot\log(R\cdot G))$; and the deficits are then filled in one pass over the sorted entries, in $O(R\cdot G)$. The step therefore costs $O(R\cdot G\cdot\log(R\cdot G))$, which the coarse-search term absorbs for T_b=$\Omega(\log(R\cdot G))$.

Summing these costs gives the stated time complexity $O(G\log G+T_b\cdot R\cdot G+T_r)$. The space bound follows from keeping a constant number of $R\times G$ matrices together with their row and group statistics, i.e., $O(R\cdot G)$, which dominates the $O(R+G)$ boundary lists and group mappings.

VI. Query Workload Generation

Through trial-and-error, instantiating queries to meet a target hardness suffers from a combinatorial search space and costly validation. To address this, GRIFFIN builds a syntactically valid template $\widehat{Q}$ from a SQL-grammar abstract syntax tree following established practice [42], and instantiates it with a **surrogate-guided query workload generation** method. Specifically, it first designs a **GA-based predicate parameter search** that finds appropriate predicate parameters with **surrogate-based fitness evaluation**, whose deviation from the target selectivity serves as the feedback for each search iteration (§ VI-A); it then designs a **region density-guided initialization** that uses the surrogates to estimate the filter density of each region, so that promising query vectors are sampled from regions whose densities match the target GLS; and it finally applies a **simulated annealing-based search** over the sampled vectors until the target GLS is reached (§ VI-B).

A. Filter Predicate Instantiation

Suppose the filter predicate in $\widehat{Q}$ contains m parameters, denoted by θ=$(\theta_1,\dots,\theta_m)$. Each parameter specifies either a scalar constant or an endpoint of a range predicate. Instantiating θ in the template yields a concrete filter predicate F_θ. Let $\sigma_g(F_\theta)$ denote its global selectivity. Given a target

global selectivity σ_g^*, GRIFFIN defines the selectivity error as $\mathrm{err}_{\mathrm{sel}}(\theta)$=$|\sigma_g(F_\theta) - \sigma_g^*|$. For a target selectivity interval $[\sigma_{g,\min}^*, \sigma_{g,\max}^*]$, the error becomes the distance from $\sigma_g(F_\theta)$ to the interval, i.e., zero if it falls inside and otherwise the gap to the nearer endpoint. Therefore, the filter predicate instantiation aims to find θ^*= $\arg\min_\theta \mathrm{err}_{\mathrm{sel}}(\theta)$.

Given a parameterized predicate $\widehat{F}$, GRIFFIN aims to find a parameter assignment θ that satisfies the target global selectivity. A naive approach randomly enumerates candidate assignments and executes each predicate in the database to evaluate its selectivity [23], [24], repeating until a satisfactory assignment is found or the search budget is exhausted. However, this approach suffers from two limitations. First, randomly enumerating parameter assignments is inefficient because the search space grows rapidly with the number of predicate parameters and their value domains. Histogram-based estimation [18] and binary search can improve efficiency for simple predicates. For example, given $A_1 < \theta_1$, a histogram can roughly estimate the value of θ_1 that achieves a target selectivity, and binary search over the attribute domain can further refine it. However, these strategies become difficult to apply to compound predicates involving $\wedge$ and $\vee$ (e.g., $A_1 < \theta_1 \wedge A_2$ `in` $\{\theta_2, \theta_3, \ldots\} \vee (A_3 \geq \theta_4)$), since the overall selectivity depends not only on individual predicate parameters but also on the intersections and unions of their qualified tuples: relaxing one predicate may select additional tuples that are still excluded by other predicates, leaving the overall selectivity unchanged. Such complex dependencies create a large combinatorial search space, making it difficult to determine an effective search direction for adjusting individual parameters toward the target selectivity using simple search strategies. Second, evaluating each candidate requires executing a database query; repeated execution over large datasets consumes considerable resources and may incur high monetary costs [16]. To address these limitations, GRIFFIN proposes to combine GA-based parameter search with surrogate-based selectivity evaluation: the former explores promising parameter combinations without requiring a predefined search direction, while the latter avoids repeated database execution.

GA-based predicate parameter search. Adjusting predicate parameters individually to reach a target selectivity is difficult because their joint effects may be highly non-linear and provide no predefined search direction. GA has been widely adopted to address this kind of problem by maintaining a population of complete parameter assignments, allowing it to search the coupled parameter space globally through evolutionary operators (i.e., *selection*, *crossover*, and *mutation*) [43]. Therefore, GRIFFIN employs GA to search for predicate parameters, as detailed in Alg. 2. Each individual represents a complete parameter assignment θ, and a population $\mathcal{P}$ consists of multiple individuals, where $|\mathcal{P}|$ denotes the population size. The fitness score of each individual is determined by its selectivity error $\mathrm{err}_{\mathrm{sel}}(\theta)$, where a smaller error indicates a better parameter assignment. Given a predicate template $\widehat{F}$ and a target selectivity interval $[\sigma_{g,\min}^*, \sigma_{g,\max}^*]$, GRIFFIN first randomly initializes a population $\mathcal{P}$ of candidate parameter

Algorithm 2: Filter Predicate Instantiation

Input: Predicate template $\widehat{F}$; target selectivity interval $[\sigma_{g,\min}^*, \sigma_{g,\max}^*]$; maximum generations T_F; error tolerance γ_s; crossover probability p_c; mutation probability p_m
Output: Parameter assignment θ^*

```
1  P ← InitializePopulation(F̂)
2  θ* ← ∅; ε_σ* ← +∞
3  for t ← 1 to T_F do
4      foreach θ ∈ P do
5          PK(F_θ) ← EvaluatePredicatePKs(F_θ)
6          σ_g(F_θ) ← |PK(F_θ)|/N; ε_σ ← err_sel(θ)
7          if ε_σ < ε_σ* then θ* ← θ; ε_σ* ← ε_σ;
8      if ε_σ* ≤ γ_s then break;
9      P ← Evolve(P, p_c, p_m)
10 return θ*
```

assignments (Line 1) and initializes the best assignment θ^* and its minimum selectivity error $\varepsilon_\sigma{}^*$ (Line 2). In each generation (Lines 3–9), it evaluates each individual θ by identifying the qualified primary keys $\mathrm{PK}(F_\theta)$ of the instantiated predicate F_θ, computing its global selectivity $\sigma_g(F_\theta)$, and obtaining its selectivity error ε_σ= $\mathrm{err}_{\mathrm{sel}}(\theta)$ (Lines 5–6). The best assignment θ^* and its error $\varepsilon_\sigma{}^*$ are updated whenever an individual achieves a smaller error (Line 7). If the best error $\varepsilon_\sigma{}^*$ is no larger than the error tolerance γ_s, the search terminates (Line 8). Otherwise, the population evolves through *selection*, *crossover*, and *mutation* (Line 9). Specifically, *selection* picks individuals with the smallest selectivity error as parents; *crossover* exchanges parameter values between two parents with probability p_c; and *mutation* perturbs one parameter of an individual by a random amount within its valid value domain with probability p_m. The best assignment θ^* is returned when the tolerance γ_s is satisfied or the maximum number of generations T_F is reached (Line 10).

Surrogate-based fitness evaluation. Although GA can efficiently explore the parameter space, it requires repeated fitness evaluations. To reduce this overhead, GRIFFIN leverages deterministic attribute mapping functions and metadata maintained during data generation to compute filter selectivity exactly, without executing SQL queries. Specifically, for each predicate literal ℓ in F_θ, GRIFFIN first identifies the satisfying attribute groups using the corresponding attribute mapping function. For example, with $f_{A_i}(j)$=$2j+1$ over group IDs $j \in \{1, \ldots, 100\}$, the predicate $A_i < \theta_1$ with θ_1=101 becomes $2j+1 < 101$, i.e., group IDs $\{1, \ldots, 49\}$, which GRIFFIN maps to PK intervals. If PK remapping has been performed, it further uses the assignment matrix to locate the reassigned PKs. Let $\mathrm{PK}(\ell)$ denote the resulting set of PKs satisfying ℓ. For a compound predicate, its qualified PK set is recursively derived through set intersection, union, and complement according to its Boolean structure. Thus, $\sigma_g(F_\theta)$=$|\mathrm{PK}(F_\theta)|/N$ is computed through in-memory calculation, avoiding expensive SQL execution.

Complexity. The search takes $O(T_F \cdot |\mathcal{P}| \cdot C_F)$ time, where C_F is the cost of evaluating a candidate using metadata.

A fitness evaluation of one individual θ instantiates the m parameters, inverts the mapping function f_A to find the attribute

groups that satisfy each predicate, converts those groups into their PK intervals, combines the intervals according to the Boolean structure of the template, and counts the resulting keys to obtain $\sigma_g(F_\theta)$ and hence $\mathrm{err}_{\mathrm{sel}}(\theta)$. All of these operations are in-memory, and their total cost is C_F; because the evaluation reads every parameter and visits the groups selected by each predicate, C_F grows with m but does not depend on N.

One generation evaluates the $|\mathcal{P}|$ individuals, costing $O(|\mathcal{P}| \cdot C_F)$, and then applies selection, crossover, and mutation. Selection scans the fitness values, and crossover and mutation copy or overwrite $O(m)$ parameter values per offspring, so evolving the population costs $O(|\mathcal{P}| \cdot (m + \log |\mathcal{P}|))$. Including initialization of the first population, which costs $O(|\mathcal{P}| \cdot m)$, the search performs at most T_F generations and therefore

$$O\big(|\mathcal{P}| \cdot m + T_F \cdot |\mathcal{P}| \cdot (C_F + m + \log |\mathcal{P}|)\big)$$

operations. Both m and $|\mathcal{P}|$ are parameters of the algorithm rather than of the data, so the evaluation term is the one that grows with the input, and the bound is stated as $O(T_F \cdot |\mathcal{P}| \cdot C_F)$.

B. Query Vector Instantiation

Once a predicate F satisfying the target global selectivity is instantiated, GRIFFIN chooses a query vector $\mathbf{q}$ to meet the target GLS, denoted $\mathrm{GLS}(\mathbf{q}, F, K)$. For a scalar target GLS^*, the GLS error is defined as $\mathrm{err}_{\mathrm{GLS}}(\mathbf{q}) = |\mathrm{GLS}(\mathbf{q}, F, K) - \mathrm{GLS}^*|$. For a target range $[\mathrm{GLS}^*_{\min}, \mathrm{GLS}^*_{\max}]$, $\mathrm{err}_{\mathrm{GLS}}(\mathbf{q})$ is the distance from $\mathrm{GLS}(\mathbf{q}, F, K)$ to the range, i.e., zero if it falls inside and otherwise the gap to the nearer endpoint; the goal is thus $\mathbf{q}^* = \arg\min_{\mathbf{q}} \mathrm{err}_{\mathrm{GLS}}(\mathbf{q})$. However, unlike selectivity, which is determined by the marginal distribution of attribute values, GLS depends on the relative spatial positions and local density of the generated vectors; it therefore cannot be derived from the key-based mapping functions, and instead requires an actual vector similarity search to identify the nearest neighbors of each candidate query vector. Gradient-based methods are unsuitable for directly optimizing the non-differentiable GLS objective [44]. Meanwhile, exhaustive/grid search is too expensive, and random sampling rarely hits the feasible region. Prior work has shown that simulated annealing is effective for searching high-dimensional vector spaces for query vectors with desired properties [45]. It iteratively perturbs the current query vector, accepting improving candidates while occasionally accepting worse ones to escape local optima [46]. However, it is sensitive to the initial query vector: a random start may lie far from the region likely to satisfy the GLS constraint, incurring expensive candidate evaluations. To address this limitation, GRIFFIN leverages the region-level metadata constructed during data generation to identify promising starting points.

Region density-guided initialization. GRIFFIN leverages region-level filter density to identify promising initial query vectors (i.e., starting points), based on two properties of the generated data. First, since each region groups spatially close vectors, the top-K nearest neighbors of a query vector are more likely to lie within its own region or adjacent ones [47]. Second, the assignment matrix fixes the number of tuples from each

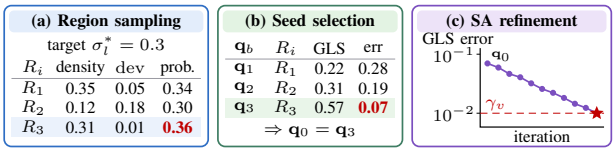


Fig. 3. Example of query vector instantiation.

attribute group assigned to each vector region, while individual tuples are randomly assigned to vectors within that region. This random assignment decouples a tuple's filter predicate from its spatial location within the region. Therefore, if a query vector's top-K nearest neighbors all lie within region R_i, its expected local selectivity equals the filter density of R_i. Given an instantiated F, its $\sigma_g(F)$ is fixed, so the target GLS maps to a target local selectivity. Therefore, regions whose filter densities are close to the target local selectivity are more likely to contain promising starting points. However, this approximation may be inaccurate when top-K neighborhoods cross region boundaries. Thus, GRIFFIN uses region-level filter density only to guide candidate query vector sampling, rather than directly evaluating GLS. Let $\mathrm{PK}(F)$ denote the set of PKs satisfying F. For each vector region R_i, GRIFFIN computes its filter density $h_F(R_i) = \frac{|\mathrm{PK}(F) \cap \mathcal{I}_i^{\mathrm{vec}}|}{|\mathcal{I}_i^{\mathrm{vec}}|}$, where $\mathcal{I}_i^{\mathrm{vec}}$ denotes the PK interval associated with R_i. Note that the target local selectivity interval can be derived from the target GLS via Eq. 2. We denote the target local selectivity interval by $[\sigma^*_{l,\min}, \sigma^*_{l,\max}]$, which reduces to a single point σ_l^* for a scalar target. Similar to the GLS error above, GRIFFIN defines the deviation $\mathrm{dev}(R_i)$ of $h_F(R_i)$ from the target interval (zero if inside, else the distance to the nearer endpoint); a smaller deviation indicates a filter density closer to the target.

To prioritize regions with smaller deviations while preserving spatial diversity, GRIFFIN assigns each region R_i a sampling weight $\omega_i = \exp(-\mathrm{dev}(R_i))$, giving higher sampling probabilities to regions with smaller deviations while keeping all regions as candidates. Alg. 3 summarizes the procedure. GRIFFIN samples n_0 candidates by selecting a region R_i with probability proportional to its weight $\omega_i \in \mathcal{W}$ and a query vector from it, evaluates each candidate's GLS error via exact top-K search (Lines 1–2), initializes both the current state and the best solution to the best candidate $(\mathbf{q}_0, \varepsilon_{v,0})$ (Line 3), and terminates if $\varepsilon_v^* \le \gamma_v$ (Line 4).

Simulated annealing-based query vector refinement. If the condition in Line 4 is not met, GRIFFIN employs simulated annealing to refine the query vector toward the target GLS constraint from $\mathbf{q}_0$ (Lines 5–10). At each iteration of the remaining evaluation budget $T_q - n_0$, where T_q is the total evaluation budget including n_0, it perturbs the current query vector $\mathbf{q}$ along a sampled direction by a random distance to generate a candidate $\mathbf{q}'$, following the standard perturbation strategy [45] (Line 6). The GLS error of $\mathbf{q}'$ is then evaluated through an exact top-K search, i.e., $\varepsilon'_v = \mathrm{err}_{\mathrm{GLS}}(\mathbf{q}')$ (Line 6). A candidate that improves the error, i.e., $\Delta\varepsilon_v = \varepsilon'_v - \varepsilon_v \le 0$, is always accepted; otherwise it is accepted with probability $\exp(-\Delta\varepsilon_v / \tau_t)$, where $\tau_t = \tau_0 / t$ is the temperature at iteration t (Line 10). As t increases, τ_t gradually decreases, so the search is more willing to accept worse candidates in early

Algorithm 3: Query Vector Instantiation

Input: Filter predicate F; target local selectivity interval $[\sigma^*_{l,\min}, \sigma^*_{l,\max}]$; seed pool size n_0; evaluation budget T_q; initial temperature τ_0; vector regions $\mathcal{R}$; error tolerance γ_v

Output: Query vector $\mathbf{q}^*$

1 $\mathcal{W} \leftarrow$ ComputeRegionWeights$(F, \mathcal{R})$
2 $\{\mathbf{q}_b\}_{b=1}^{n_0} \leftarrow$ WeightedSample$(\mathcal{R}, n_0, \mathcal{W})$; $(\mathbf{q}_0, \varepsilon_{v,0}) \leftarrow$ SelectSeed$(\{\mathbf{q}_b\}_{b=1}^{n_0}, n_0)$;
3 $(\mathbf{q}, \varepsilon_v) \leftarrow (\mathbf{q}_0, \varepsilon_{v,0})$; $(\mathbf{q}^*, \varepsilon_v^*) \leftarrow (\mathbf{q}_0, \varepsilon_{v,0})$;
4 **if** $\varepsilon_v^* \le \gamma_v$ **then return** $\mathbf{q}^*$;
5 **for** $t \leftarrow 1$ **to** $T_q - n_0$ **do**
6 $\quad \mathbf{q}' \leftarrow$ LocalPerturb$(\mathbf{q})$; $\varepsilon_v' \leftarrow \mathrm{err}_{\mathrm{GLS}}(\mathbf{q}')$;
7 $\quad$ **if** $\varepsilon_v' < \varepsilon_v^*$ **then** $(\mathbf{q}^*, \varepsilon_v^*) \leftarrow (\mathbf{q}', \varepsilon_v')$;
8 $\quad$ **if** $\varepsilon_v^* \le \gamma_v$ **then return** $\mathbf{q}^*$;
9 $\quad \Delta\varepsilon_v \leftarrow \varepsilon_v' - \varepsilon_v$;
10 $\quad$ **if** $\Delta\varepsilon_v \le 0$ ***or*** Random$() < \exp(-\Delta\varepsilon_v/(\tau_0/t))$ **then** $(\mathbf{q}, \varepsilon_v) \leftarrow (\mathbf{q}', \varepsilon_v')$;
11 **return** $\mathbf{q}^*$;

iterations to escape local optima, and later focuses on reducing $\mathrm{err}_{\mathrm{GLS}}$. Throughout refinement, GRIFFIN updates $\mathbf{q}^*$ whenever a candidate achieves a smaller $\mathrm{err}_{\mathrm{GLS}}$ (Line 7). The search terminates when the target constraint is satisfied (Line 8), or the evaluation budget is exhausted.

For example, in Fig. 3, suppose $\sigma_g(F)$=0.1, a target GLS*=0.5, the target local selectivity σ_l^*=0.3 and n_0=3. In Fig. 3(a), GRIFFIN assigns the largest normalized sampling weight (0.36) to R_3, whose filter density (0.31) is closest to the target. In Fig. 3(b), the n_0=3 sampled candidates have exact GLS errors of 0.28, 0.19, and 0.07, respectively. Thus, $\mathbf{q}_3$ is selected as the initial seed, i.e., $\mathbf{q}_0$=$\mathbf{q}_3$. Finally, simulated annealing refines $\mathbf{q}_0$ to obtain $\mathbf{q}^*$, as shown in Fig. 3(c).

Complexity. Since the n_0 initial candidates and the at most $T_q - n_0$ annealing steps are all exact evaluations, the overall time complexity is $O\left(|\mathcal{R}| \cdot C_R + T_q \cdot C_{\mathrm{exact}}\right)$, where C_R and C_{exact} denote the costs of computing a region's filter density and performing an exact top-K search, respectively.

Once F is fixed, the density $h_F(R_i)$=$|\mathrm{PK}(F) \cap \mathcal{I}_i^{\mathrm{vec}}|/|\mathcal{I}_i^{\mathrm{vec}}|$ follows from the PK interval of the region and the qualified key set: intersecting the two and counting the keys costs C_R per region. The deviation from the target interval and the weight ω_i= $\exp(-\mathrm{dev}(R_i))$ are then computed in $O(1)$ per region, and normalizing the weights costs $O(|\mathcal{R}|)$. The weights are cumulated once, so sampling a region costs $O(\log|\mathcal{R}|)$ and drawing the n_0 seeds costs $O(n_0 \cdot \log|\mathcal{R}|)$; each seed vector is read in $O(d)$.

Initialization evaluates the GLS error of the n_0 seeds, and each annealing step evaluates one perturbed vector, so the search performs exactly $n_0 + (T_q - n_0)$=T_q exact top-K searches, whose scores are reused when the best candidate is updated. One search retrieves the K nearest neighbors of the candidate and tests each neighbor against F, which costs C_{exact}; a perturbation costs $O(d)$. Adding the density computation and the evaluations gives

$$O\big(|\mathcal{R}| \cdot C_R + T_q \cdot C_{\mathrm{exact}}\big),$$

since a single distance computation over the N vectors of dimension d already dominates both the $O(n_0 \cdot \log|\mathcal{R}|)$ sampling and the $O(T_q \cdot d)$ perturbations.

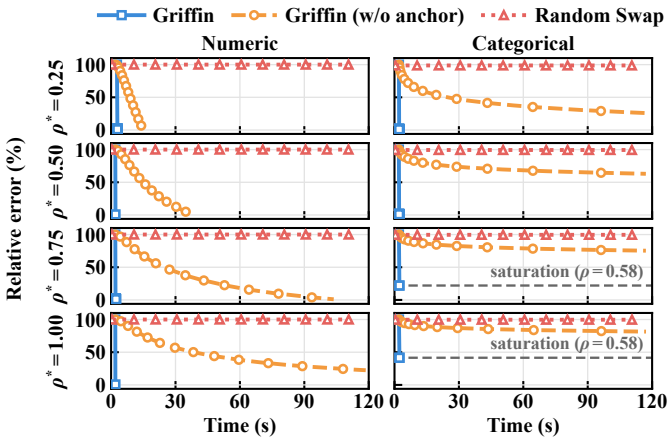


Fig. 4. Comparison of data generation accuracy.

VII. Experiments

A. Experimental Setup

Experimental environment. We deploy GRIFFIN, implemented in Python, on a machine running TencentOS Server 3.2 with an AMD EPYC 9K65 processor (32 logical CPUs), 64 GB of RAM, and 500 GB of disk storage.

Baselines. For data generation, to the best of our knowledge, no existing method supports controlling attribute–vector dependency; we therefore compare GRIFFIN with two custom-designed baselines: Random Swap, which applies random tuple-level swaps starting from independent generation, and GRIFFIN (w/o anchor), which drops the coarse-search stage but keeps the assignment matrix and refinement stage.

For query generation, we compare GRIFFIN with three representative baselines and one ablation variant. SQLSmith [23] serves as a random-search baseline that repeatedly samples parameter assignments; the LLM-based baseline adopts established prompt engineering practices [26], [48], prompting DeepSeek-V4-Flash (one of the most advanced open-source LLMs at the time of our experiments) with the schema, target selectivity, and GLS constraints to directly infer the predicate parameters and query vector. VecBench [18] generates single-attribute predicates by sampling a query vector, identifying its nearest neighbors, and fitting predicates to their attribute distributions; we adapt it to multi-predicate templates by assuming predicate independence and decomposing the target selectivity into per-predicate selectivities. These three baselines execute the generated queries on pgvector to obtain execution errors and determine their satisfactions. GRIFFIN (w/o guided) is an ablation that removes the region density-guided initialization (§ VI-B) while keeping all other components unchanged.

Benchmark configuration. For data generation, we generate a synthetic table containing 1M tuples, with one 192-dimensional vector column by default, following the settings of the representative YFCC dataset [49], [50] used in VecBench [18], and 21 scalar attributes following the widely used relational DMV dataset [51], including 11 numerical (*continuous*) and 10 categorical (*discrete*) columns that support numerical range and categorical equality predicates. For each scalar column,

GRIFFIN randomly selects one of six distribution settings, parameterized by $s \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$, where s=0 corresponds to the uniform distribution and $s > 0$ to Zipfian distributions with different skewness. The numbers of attribute groups G=$|\mathcal{G}|$ and vector regions R=$|\mathcal{R}|$ are independently selected from $\{10^2, 10^3, 10^4\}$ and $\{16, 32, 64, 128, 256\}$, respectively. The target dependency strength ρ^* between each scalar column and the vector column is randomly sampled from $[0, 1]$. By default, we generate vectors using a Gaussian mixture model [52] with 32 clusters whose sizes follow a Zipfian distribution. For query generation, each FANNS query consists of one filter predicate and a query vector, retrieving the top-K qualifying neighbors with K=10 by default [13]. Global selectivity σ_g and GLS are used to define query hardness. Following VecBench [18], we categorize global selectivity into low (0%–1%), medium (1%–50%), and high (50%–100%), and similarly categorize GLS into low (GLS < -0.3), medium ($-0.3 \leq$ GLS ≤ 0.3), and high (GLS > 0.3), following Amanbayev et al. [13]. Data and query generation are each allocated a time budget of 120 s, which is converted into iteration limits for data-side coarse search (T_b) and refinement (T_r), as well as query-side predicate search (T_F) and vector search (T_q), based on their respective average per-iteration execution times. In contrast, KMeans training uses a fixed iteration limit of T_k=64, independent of the time budget.

B. Overall Evaluation

1) Effectiveness of Data Generation: We evaluate the accuracy and efficiency of GRIFFIN in achieving target attribute–vector data dependencies. We randomly select one numerical and one categorical attribute, vary the target dependency strength $\rho^* \in \{0.25, 0.50, 0.75, 1.00\}$ for each, and measure the relative error within the specified time budget. For the numerical attribute, Fig. 4 shows that GRIFFIN rapidly reduces the relative error below 1% for all four target ρ^*. This efficiency stems from its coarse-to-fine search strategy, which first binary-searches interpolations between the low- and high-dependency anchors for a promising initial assignment, then refines the assignment locally to minimize the remaining dependency error. Without anchor interpolation, GRIFFIN (w/o anchor) requires 103.71 s to reach ρ^*=0.75 (45.5× longer), while its error for ρ^*=1.00 remains 22.21% after 120 s. Random Swap, by contrast, fails to approach any target. For the categorical attribute, GRIFFIN achieves errors below 1% for the two lower target ρ^* but saturates at ρ=0.5847 for both ρ^*=0.75 and ρ^*=1.00. It approaches the theoretical maximum dependency strength of approximately 0.59, constrained by the specified attribute distributions and number of vector regions.

2) Scalability of Data Generation: We evaluate the scalability of GRIFFIN in terms of data generation time, metadata storage overhead, and PK remapping latency. We independently vary the data size N, vector dimensionality d, the number of regions R, and the number of attribute groups G. We use N=10^6, d=192, G=1000, R=32, and ρ^*=0.25 as the default configuration, varying one factor at a time while keeping the others fixed. Note that we break down the generation time into five stages, which are relational attribute generation (Attr Gen), vector generation (Vector Gen), vector region partitioning (KMeans), coarse-search stage, and refinement stage.

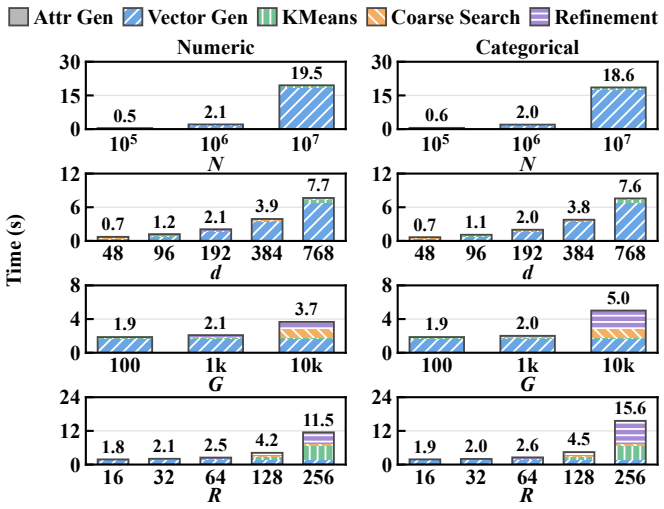


Fig. 5. Scalability of data generation.

Data size N and dimension d. As shown in Fig. 5, increasing N and d primarily affects vector generation. Taking numerical attributes as an example, as N increases from 10^5 to 10^6 and 10^7, vector generation time increases from 0.17 to 1.71 and 18.51 s, respectively. Similarly, as d increases from 48 to 768, the time grows from 0.43 to 6.83 s. Both exhibit nearly linear growth, consistent with the $O(N \cdot d)$ cost of generating one d-dimensional vector per tuple. Vector region partitioning uses FAISS KMeans [53], which trains on at most $256R$ sampled vectors, bounding the centroid fitting cost to $O(\min(N, 256R) \cdot R \cdot d \cdot T_k)$. This cost is substantially lower than that of vector generation, which must materialize all N vectors. Meanwhile, attribute generation ($O(G)$) and dependency control (on an $R \times G$ matrix) incur nearly constant overhead as N and d increase. Note that the reported time excludes disk I/O, such as writing generated data to CSV files.

Number of attribute groups G. Fig. 5 shows that increasing G primarily affects coarse search and refinement. For numerical attributes, as G increases from 10^2 to 10^3 and 10^4, coarse search time increases from 0.007 to 0.063 and 1.016 s, respectively, exhibiting an overall superlinear growth trend. Coarse search has a time complexity of $O(T_b \cdot R \cdot G)$. Although the per-trial cost grows linearly with G, the number of trials to satisfy the tolerance varies with the matrix size, explaining the superlinear trend. Similarly, refinement time also increases from 0 to 0.17 and 0.77 s, as larger assignment matrices introduce greater integerization errors, requiring more cycle moves to meet the target dependency. In contrast, vector generation is independent of G, while attribute generation exhibits only a slight increase in runtime. This is because attribute generation mainly partitions the fixed set of N primary keys into G groups and constructs their boundaries and value mappings in $O(G)$ time, incurring minimal metadata construction overhead.

Number of regions R. As shown in Fig. 5, increasing R primarily affects KMeans clustering and refinement. For numerical

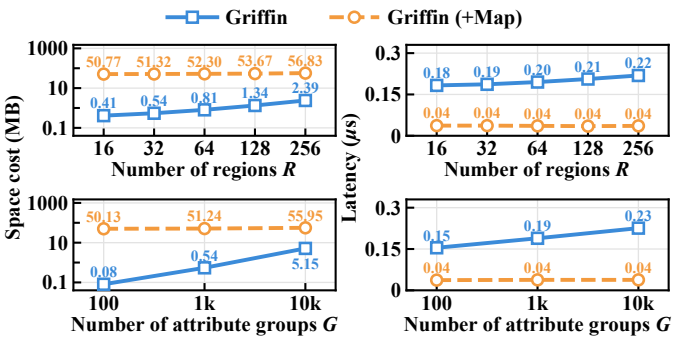
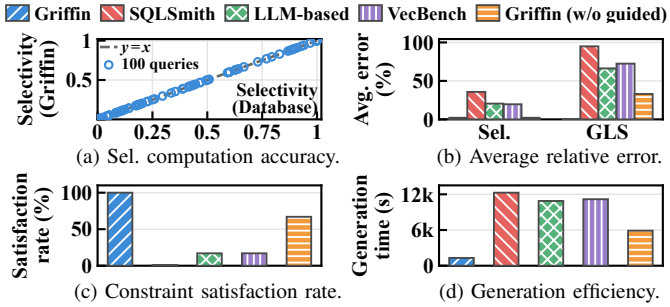


Fig. 6. Space cost and lookup latency analysis.



(a) Sel. computation accuracy. (b) Average relative error.

(c) Constraint satisfaction rate. (d) Generation efficiency.

Fig. 7. Accuracy and efficiency of query generation.

attributes, as R increases from 16 to 128 and 256, KMeans time increases from 0.083 to 1.324 and 5.226 s, respectively, exhibiting a superlinear growth trend. This is because KMeans has a time complexity of $O(\min(N, 256R) \cdot R \cdot d \cdot T_k)$, which grows quadratically with R when $N \geq 256R$ and T_k is fixed. However, larger R tends to require fewer iterations for convergence, partially offsetting the increased per-iteration cost and resulting in superlinear but subquadratic runtime growth. Refinement time likewise grows to 4 s, as larger assignment matrices need more cycle moves to satisfy the error tolerance.

Space cost and PK remapping latency. We first evaluate the memory consumption of GRIFFIN's metadata. Note that GRIFFIN maintains the two boundary lists $\mathcal{B}^{\text{vec}}$ and $\mathcal{B}^{\text{attr}}$, the mapping functions f_A, and the assignment matrix M, with a total space complexity of $O(R+G+R \cdot G)$. We vary R and G to measure metadata size, comparing GRIFFIN with GRIFFIN (+Map), which additionally maintains a per-tuple mapping table for aligned tuples; memory consumption is measured using Python's *asizeof* API [54] on the in-memory structures. Fig. 6 shows that GRIFFIN's metadata grows from 0.41 to 2.39 MB as R increases from 16 to 256, and from 0.081 to 5.15 MB as G increases from 100 to 10,000, exhibiting sublinear growth. At R=32, it occupies only 0.54 MB versus 51.32 MB for GRIFFIN (+Map), a 98.95% reduction. We further evaluate the cost of resolving a PK using the *Boundary-aware Mapping* strategy (§ V-B). Fig. 6 shows that GRIFFIN (+Map) resolves a PK via a single hash-table lookup in nearly 0.04 μs ($O(1)$), whereas GRIFFIN performs two binary searches over region and group boundaries to determine the PK's attribute group, taking 0.15–0.23 μs ($O(\log R + \log G)$). Despite the slightly higher lookup latency, both methods achieve sub-microsecond performance, making the overhead negligible during parameter search.

3) Effectiveness of Query Generation: We evaluate two aspects of GRIFFIN's query generation: (i) the correctness of surrogate-based selectivity computation and (ii) the accuracy and efficiency of generating queries that satisfy target selectivity and GLS constraints. We generate 100 queries, each containing 1–5 predicates over randomly selected attributes and comparison operators, combined via AND/OR. For correctness verification, we randomly instantiate predicate parameters and compare the surrogate-computed selectivity with that measured on PostgreSQL v16.14 with pgvector v0.8.0; since selectivity and GLS depend on data and query semantics rather than the execution engine, one system is sufficient. For accuracy and efficiency evaluation, we generate an equal number of queries per global selectivity level (low, medium, and high); all methods share the same query templates and differ only in their instantiation strategies. For each query, we precompute the feasible GLS range from the data distribution and randomly pick a target within it to avoid infeasible configurations; a 10% relative-error threshold applies to both constraints, and failed instantiations (e.g., timeouts) are charged a 100% error.

Selectivity verification. Fig. 7(a) shows that the surrogate-computed and pgvector-measured selectivities match exactly across all queries, with a maximum absolute error of 0. This is because GRIFFIN uses the deterministic surrogates to identify all qualified PKs, enabling exact selectivity computation.

Accuracy and generation efficiency. GRIFFIN achieves average errors below 10% for both selectivity and GLS, substantially outperforming all baselines and its variant without region density-guided initialization, as shown in Fig. 7(b). As shown in Fig. 7(c), GRIFFIN satisfies both constraints for all queries (100% satisfaction), whereas the second-best variant without region density-guided initialization succeeds on only 67%; as shown in Fig. 7(d), GRIFFIN completes in 1,305 s, about 10× faster than the three baselines and nearly 5× faster than the variant. These improvements stem from GRIFFIN's surrogate-guided query workload generation, which efficiently identifies parameters meeting the target selectivity and query vectors achieving the target GLS. In contrast, SQLSmith relies on random search, with a large search space and expensive validation; the LLM-based baseline incurs substantial inference overhead and struggles to infer the relationship between parameters, selectivity, and GLS from distribution information alone; VecBench's distribution-based estimates degrade on compound predicates and ignore GLS; the variant without region density-guided initialization ignores the spatial distribution of qualifying vectors, requiring more search effort and yielding lower satisfaction rates.

4) Hardness Validation: We assess whether GRIFFIN's selectivity and GLS controls produce distinguishable query hardness across three widely adopted FANNS systems (pgvector, FAISS, and Milvus), using both graph-based HNSW and partition-based IVF indexes. Specifically, pgvector v0.8.0, FAISS v1.13.2, and Milvus v2.5.4 represent three distinct architectures: a relational-database extension, an embedded library, and a native vector database. Following prior work [13], we configure HNSW with `M`=15, the maximum number of connections per node, and `efConstruction`=75,

the candidate list size during index construction. We set `efSearch` $\in \{10, 40, 100, 200, 500, 1000\}$, which controls the search candidate list size in HNSW, and `nprobe` $\in \{5, 10, 20, 50, 150, 300\}$, which determines the number of inverted lists searched in IVF. We report top-K results with $K \in \{10, 100\}$. Each curve in Fig. 8 consists of up to six points, one per `efSearch` (HNSW) or `nprobe` (IVF) value; Milvus requires `efSearch` $\geq K$, so its HNSW curves for K=100 contain only four points. Each point uses 100 queries sharing the same target selectivity or GLS level.

Selectivity. We fix each predicate template and query vector, varying predicate parameters to achieve the three selectivity levels. Fig. 8 shows that selectivity affects both recall and QPS, with the impact varying across indexes and filtering implementations: for HNSW, lower selectivity generally leads to lower QPS at comparable recall in pgvector and FAISS. This is because qualifying vectors become sparser, so achieving the same recall requires a larger `efSearch` to find enough valid nearest neighbors. Milvus behaves differently: its low-selectivity workload achieves perfect recall across all search configurations, because it switches to brute-force search over few filtered candidates, making performance largely insensitive to `efSearch` [55]. For IVF, FAISS and Milvus generally achieve higher QPS under lower selectivity at comparable recall: their hybrid strategies integrate predicate filtering into vector search, skipping non-qualifying vectors during inverted-list scanning and thus reducing distance computations as selectivity decreases. In contrast, pgvector often achieves higher QPS under higher selectivity at matched recall, because its post-filtering strategy retrieves candidates largely independently of the predicate, and higher selectivity allows enough qualifying neighbors to be found by probing fewer lists [55]–[58]. These results demonstrate that varying selectivity induces distinct query hardness, with the direction and magnitude of changes depending on the filtering implementation.

GLS. We fix the predicates and their global selectivity, varying only the query vectors to achieve the three GLS levels. Fig. 8 shows that GLS likewise affects both recall and QPS, with the impact varying across indexes and filtering implementations. For HNSW, higher GLS generally leads to higher QPS at comparable recall across pgvector, FAISS, and Milvus. This is because higher GLS indicates a greater concentration of qualifying vectors near the query, so achieving the same recall typically requires a smaller `efSearch`, reducing graph exploration overhead. For IVF, the impact of GLS differs across filtering implementations. At comparable recall, pgvector's post-filtering strategy generally achieves higher QPS under higher GLS, as qualifying neighbors become retrievable with fewer probed lists. In contrast, FAISS and Milvus with hybrid filtering generally exhibit lower QPS under higher GLS over the moderate-recall range. However, the QPS differences across GLS levels tend to narrow as `nprobe` increases: IVF search cost depends not only on the number of probed lists but also on the qualifying candidates processed within them, and for FAISS and Milvus, higher GLS increases the candidates requiring distance computation; as more lists are probed, the relative

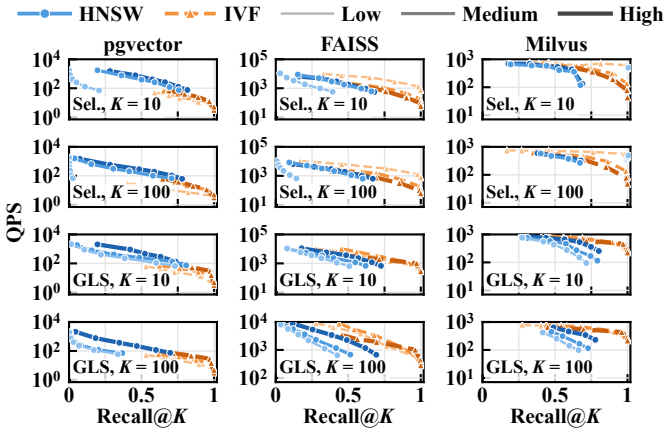


Fig. 8. Recall–QPS trade-offs under varying sel. and GLS.

impact of this additional candidate-processing cost diminishes, narrowing the gap [55]–[58]. These results demonstrate that varying GLS induces distinct query hardness even at fixed global selectivity, with the impact depending on the index structure and filtering implementation.

5) Case Study of Performance Anomalies: We present two cases where GRIFFIN's controllable workload generation reveals performance anomalies overlooked by existing work: **1) Performance anomaly under complex predicates.** Using GRIFFIN, we observed a severe performance anomaly in Milvus when evaluating a complex predicate. Specifically, a generated query with σ_g=20% and 19 conjunctive conditions, some of which are tautologies, runs at 30.80 ms on Milvus versus only 0.047 ms on FAISS (a gap of over 600×). A tautological condition (e.g., a range covering an attribute's full domain) needs no evaluation once the system knows the attribute's min–max range, yet Milvus still evaluates it instead of simplifying $F \wedge \text{true}$ into F; we reported this, and the Milvus team confirmed it (Issue #54195). Such an anomaly surfaces only under complex multi-predicate queries; existing work generates only single predicates and thus cannot expose it, whereas GRIFFIN's support for predicates of arbitrary complexity enables this diagnosis. **2) Performance anomaly under low GLS.** Using GRIFFIN, we observed a severe post-filtering anomaly on pgvector for queries with low GLS. At the same global selectivity (σ_g=25%), high-GLS queries (GLS $\in [0.33, 0.60]$) achieve a recall of 0.77–0.97, whereas low-GLS queries (GLS $=-1.0$) return no results across all `efSearch` values. The anomaly stems from pgvector's fixed post-filtering strategy: it always retrieves candidates from HNSW neighborhoods around the query, which for low-GLS queries contain no qualifying tuples. By generating workloads with arbitrary combinations of GLS and global selectivity, GRIFFIN can support the evaluation and tuning of adaptive filtering strategies that depend on these two factors.

VIII. CONCLUSION

This paper presents GRIFFIN, a customizable framework for generating attribute–vector datasets and hardness-targeted FANNS query workloads. Extensive experiments demonstrate its high accuracy, efficiency, scalability, and practical utility.

REFERENCES

[1] Z. Hua, Q. Mo, Z. Yao, L. Cui, X. Liu, G. Wang, Z. Wei, X. Liu, T. Tang, S. Liu, and L. Qu, "Dynamically detect and fix hardness for efficient approximate nearest neighbor search," in *SIGMOD*, vol. 3, no. 6, 2025, pp. 1–28.

[2] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," in *arXiv:2505.06501*, 2025.

[3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," in *arXiv:2312.10997*, 2023.

[4] G. Kang, Z. Ge, J. Hu, X. Zhang, L. Wang, and J. Zhan, "BigVectorBench: Heterogeneous data embedding and compound queries are essential in evaluating vector databases," in *PVLDB*, vol. 18, no. 5, 2025, pp. 1536–1550.

[5] X. Zhao, X. Zhou, and G. Li, "Chat2Data: An interactive data analysis system with RAG, vector databases and LLMs," in *PVLDB*, vol. 17, no. 12, 2024, pp. 4481–4484.

[6] J. Tan, J. Hu, and S. Dong, "Incorporating entity-level knowledge in pretrained language model for biomedical dense retrieval," in *Computers in Biology and Medicine*, vol. 166, 2023, p. 107535.

[7] W. Wang, Y. Guo, C. Shen, S. Ding, G. Liao, H. Fu, and P. K. Prabhakar, "Integrity and junkiness failure handling for embedding-based retrieval: A case study in social network search," in *SIGIR*, 2023, pp. 3250–3254.

[8] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension," in *SIGMOD*, 2020, pp. 2241–2253.

[9] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," in *PVLDB*, vol. 18, no. 10, 2025, pp. 3256–3268.

[10] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search," in *SIGMOD*, vol. 2, no. 6, 2024, pp. 1–26.

[11] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Zhang, N. Mahapatro, P. Srinivasan, A. Singh, and H. V. Simhadri, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *WWW*, 2023, pp. 3406–3416.

[12] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *ICML*, 2024, pp. 12 469–12 490.

[13] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," in *arXiv:2602.11443*, 2026.

[14] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: performant and predicate-agnostic search over vector embeddings and structured data," in *SIGMOD*, vol. 2, no. 3, 2024, pp. 1–27.

[15] J. Yang, P. Wu, G. Cong, T. Zhang, and X. He, "SAM: Database generation from query workloads with supervised autoregressive models," in *SIGMOD*, 2022, pp. 1542–1555.

[16] Z. Hu, R. Zhang, C. Yang, X. Zhou, Q. Xu, and C. Yang, "Artemis: A customizable workload generation toolkit for benchmarking cardinality estimation," in *ICDE*, 2025, pp. 4628–4631.

[17] P. Iff, P. Brügger, M. Chrapek, D. Kochergin, M. Besta, and T. Hoefler, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," in *SIGIR*, 2026, pp. 610–622.

[18] X. Zhang, C. Zhang, J. Fan, G. Li, and X. Du, "VecBench: A controllable benchmark for filtered vector search," in *SIGMOD*, vol. 4, no. 3, 2026, pp. 1–27.

[19] D. Lu, H. Caminal, M. Chatzakis, Y. Papakonstantinou, Y. Chronis, V. Jain, and F. Özcan, "An in-depth study of filter-agnostic vector search on a PostgreSQL database system," in *SIGMOD*, vol. 4, no. 3, 2026, pp. 1–26.

[20] H.-Y. Chang, P.-Y. Chen, T.-H. Chou, C.-S. Kao, H.-Y. Yu, Y.-T. Lin, and Y.-N. Chen, "A survey of data synthesis approaches," in *arXiv:2407.03672*, 2024.

[21] W. Xia, M. Yang, W. Li, and W. Wang, "E2E: Efficient filtered AKNN search via adaptive termination," in *SIGKDD*, 2026, pp. 5602–5613.

[22] Griffin, "Technical report and source code," https://github.com/DBHammer/Griffin, 2026.

[23] A. Seltenreich, "SQLsmith," https://github.com/anse1/sqlsmith, 2020.

[24] L. Zhang, C. Chai, X. Zhou, and G. Li, "LearnedSQLGen: Constraint-aware SQL generation using reinforcement learning," in *SIGMOD*, 2022, pp. 945–958.

[25] J. Lao and I. Trummer, "SQLBarber: A system leveraging large language models to generate customized and realistic SQL workloads," in *SIGMOD*, vol. 4, no. 1, 2026, pp. 1–27.

[26] T. Schmidt, V. Leis, P. Boncz, and T. Neumann, "SQLStorm: Taking database benchmarking into the LLM era," in *PVLDB*, vol. 18, no. 11, 2025, pp. 4144–4157.

[27] Q. Wang, H. Li, Z. Hu, R. Zhang, C. Yang, P. Cai, X. Zhou, and A. Zhou, "Mirage: Generating enormous databases for complex workloads," in *ICDE*, 2024, pp. 3989–4001.

[28] Z. Zhang, S. Liu, D. Xu, W. Lu, J. Leng, S. Du, and X. Du, "DBRepro: Automated database synthesis via a hybrid constraint-solving approach for reproducing slow queries," in *ASE*, 2026, pp. 3754–3766.

[29] J. Zhu, J. Yuan, K. Yang, X. Chen, S. Yu, H. Lv, Y. Li, and B. Zheng, "An experimental evaluation of hybrid querying on vectors," in *PVLDB*, vol. 19, no. 2, 2025, pp. 183–195.

[30] Z. Hu, S. Weng, Z. Pan, R. Zhang, C. Yang, P. Cai, X. Zhou, Q. Xu, and C. Yang, "Vodka: Rethink benchmarking philosophy in HTAP systems," in *PVLDB*, vol. 19, no. 3, 2025, pp. 481–494.

[31] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is "nearest neighbor" meaningful?" in *ICDT*, 1999, pp. 217–235.

[32] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[33] T. M. Cover and J. A. Thomas, "Elements of information theory," in *Wiley-Interscience*, Hoboken, NJ, 2006.

[34] E. H. Linfoot, "An informational measure of correlation," in *Information and Control*, vol. 1, no. 1, 1957, pp. 85–89.

[35] D. Lakens, "Calculating and reporting effect sizes to facilitate cumulative science: a practical primer for t-tests and ANOVAs," in *Frontiers in Psychology*, vol. 4, 2013, p. 863.

[36] P. E. Latham and Y. Roudi, "Mutual information," in *Scholarpedia*, vol. 4, no. 1, 2009, p. 1658.

[37] R. Koopman and S. Wang, "Mutual information based labelling and comparing clusters," in *Scientometrics*, vol. 111, no. 2, Feb. 2017, pp. 1157–1167.

[38] P. Fränti and S. Sieranoja, "K-means properties on six clustering benchmark datasets," in *Applied Intelligence*, vol. 48, no. 12, 2018, pp. 4743–4759.

[39] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, and A. Y. Wu, "An efficient k-means clustering algorithm: Analysis and implementation," in *TPAMI*, vol. 24, no. 7, 2002, pp. 881–892.

[40] O. Bachem, M. Lucic, and A. Krause, "Scalable k-means clustering via lightweight coresets," in *SIGKDD*, 2018, pp. 1119–1127.

[41] P. Belotti, C. Kirches, S. Leyffer, J. Linderoth, J. Luedtke, and A. Mahajan, "Mixed-integer nonlinear optimization," in *Acta Numerica*, vol. 22, 2013, pp. 1–131.

[42] M. Rigger, "SQLancer," https://github.com/sqlancer/sqlancer, 2026.

[43] J. Castelein, M. Aniche, M. Soltani, A. Panichella, and A. Van Deursen, "Search-based test data generation for SQL queries," in *ICSE*, 2018, pp. 1220–1230.

[44] Y. Xie, H. Dai, M. Chen, B. Dai, T. Zhao, H. Zha, W. Wei, and T. Pfister, "Differentiable top-k with optimal transport," in *NeurIPS*, vol. 33, 2020, pp. 20 520–20 531.

[45] M. Ceccarello, A. Levchenko, I. Ileana, and T. Palpanas, "Evaluating and generating query workloads for high dimensional vector similarity search," in *SIGKDD*, 2025, pp. 5299–5310.

[46] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," in *Statistical Science*, vol. 8, no. 1, 1993, pp. 10–15.

[47] Q. Yue, X. Xu, Y. Wang, Y. Tao, and X. Luo, "Routing-guided learned product quantization for graph-based approximate nearest neighbor search," in *ICDE*, 2024, pp. 4870–4883.

[48] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better zero-shot reasoning with role-play prompting," in *NAACL*, 2024, pp. 4099–4113.

[49] B. Thomee, D. A. Shamma, G. Friedland, B. Elizalde, K. Ni, D. Poland, D. Borth, and L.-J. Li, "YFCC100M: The new data in multimedia research," in *CACM*, vol. 59, no. 2, 2016, pp. 64–73.

[50] H. V. Simhadri, M. Aumüller, M. Douze, D. Baranchuk, A. Ingber, E. Liberty, G. Williams, B. Landrum, M. D. Manohar, M. Karjikar, L. Dhulipala, M. Chen, Y. Chen, R. Ma, K. Zhang, Y. Cai, J. Shi, W. Zheng, Y. Chen, J. Yin, and B. Huang, "Results of the Big ANN: NeurIPS'23 competition," in *NeurIPS*, vol. 38, 2025, pp. 76 394–76 408.

[51] State of New York, "Vehicle, snowmobile, and boat registrations," https://catalog.data.gov/dataset/vehicle-snowmobile-and-boat-registrations, 2020.

[52] C. M. Bishop, *Pattern Recognition and Machine Learning*, ser. Information Science and Statistics. Springer, 2006.
[53] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," in *TBDATA*, vol. 7, no. 3, 2021, pp. 535–547.
[54] Pympler Developers, "pympler.asizeof," https://pympler.readthedocs.io/en/stable/library/asizeof.html, 2026.
[55] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *SIGMOD*, 2021, pp. 2614–2627.
[56] FAISS, "Setting search parameters for one query," https://github.com/facebookresearch/faiss/wiki/Setting-search-parameters-for-one-query, 2026.
[57] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," in *VLDBJ*, vol. 33, no. 5, 2024, pp. 1591–1615.
[58] pgvector, "pgvector: Open-source vector similarity search for Postgres," https://github.com/pgvector/pgvector, 2024.